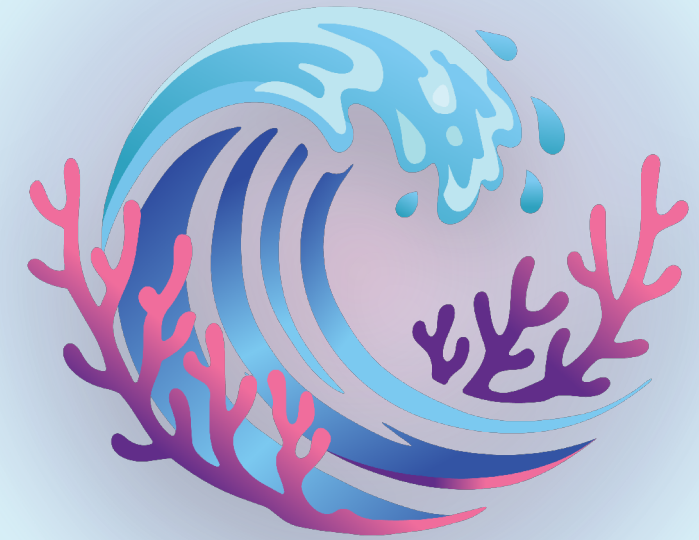




// session 08 beginner Python

More classes & morale

Give encounters their own behaviour. Let the crew's choices shape what happens next.



Build a system of things that **work together.**

Encounters

Four classes. One shared pattern:
process(ship).

Morale

A new ship attribute that changes rewards
and damage.

Game feel

Careful play helps. Reckless play can
snowball.

Last week, the ship stopped being scattered.

Name, oxygen and hull now travel together.

Ship class



attributes



methods

Prediction

What other part of the game has its own data and behaviour?

Encounters

Damage, costs and rewards already belong together.

One function decides every encounter.

```
oxygen, hull, narration = process_encounter(  
    destination, oxygen, hull  
)
```



The problem

Different encounters carry different data, but all are hidden behind one engine function.

An encounter bundles its data and behaviour.

Data

danger_level
base_damage
oxygen_cost

Behaviour

process(ship) changes the ship and returns narration.

One class owns the asteroid rule.

```
class AsteroidField:
    def __init__(self, danger_level):
        self.danger_level = danger_level
        self.base_damage = 10

    def process(self, ship):
        damage = self.base_damage + (self.danger_level * 2)
        ship.hull = ship.hull - damage
        return f"Asteroid debris hits the hull: -{damage} hull."
```

1 Create it with a danger level.

2 Calculate fixed damage.

3 Change ship.hull.

4 Hand narration back.



// predict trace the object

**A ship starts with 100 hull.
Danger is 3. What hull remains?**

Damage = $10 + (3 \times 2) = 16 \rightarrow$ hull = 84

Same shape. Different details.

```
class Raider:
    def __init__(self, danger_level):
        self.danger_level = danger_level
        self.base_damage = 12
        self.oxygen_cost = 5

    def process(self, ship):
        damage = self.base_damage + (self.danger_level * 2)
        ship.hull = ship.hull - damage
        ship.oxygen = ship.oxygen - self.oxygen_cost
        return f"Raiders attack: -{damage} hull, -{self.oxygen_cost}
oxygen."
```

The shared promise

Every encounter will offer
process(ship), even when the internal
rules differ.

Not every encounter deals damage.

```
class Trader:
    def __init__(self):
        self.oxygen_refill = 15
        self.hull_repair = 10

    def process(self, ship):
        ship.oxygen = ship.oxygen + self.oxygen_refill
        ship.hull = ship.hull + self.hull_repair
        return "A friendly trader shares supplies."

class EmptySpace:
    def process(self, ship):
        return "All is calm. The crew rests."
```

Trader

Changes oxygen and hull.

Empty space

Returns narration only.

The destination chooses what we create.

```
def create_encounter(destination):  
    encounter_type = destination["encounter"]  
    danger = destination["danger_level"]  
  
    if encounter_type == "asteroid_field":  
        return AsteroidField(danger)  
    elif encounter_type == "raider":  
        return Raider(danger)  
    elif encounter_type == "trader":  
        return Trader()  
    else:  
        return EmptySpace()
```

input destination dictionary

read encounter type + danger

return one encounter object

The loop no longer needs encounter-specific rules.

```
destination = {"encounter": "asteroid_field", "danger_level": 3}  
ship = DemoShip()  
encounter = create_encounter(destination)  
narration = encounter.process(ship)
```

Read it aloud

Create the right encounter. Ask it to process the ship. Keep the returned narration.

Three returned values become one object interaction.

```
oxygen, hull, narration = process_encounter(  
    destination, oxygen, hull  
)
```



```
encounter = create_encounter(destination)  
narration = encounter.process(ship)
```

Build the encounter system in four passes.

01

AsteroidField

02

Raider

03

Trader + EmptySpace

04

create_encounter

RUN AFTER EVERY PASS Keep the game working while the engine fades out.



// chapter two a new ship stat

**Two crews face the same danger.
Should they always get the same
result?**

Morale lets past choices influence the next encounter.

Morale changes the strength of outcomes.

80

less help
more damage

100

normal result

120

more help
less damage

One formula boosts help. One boosts harm.

```
def positive_amount(base_amount, morale):  
    return int(base_amount * morale / 100)  
  
def negative_amount(base_amount, morale):  
    return int(base_amount * (200 - morale) / 100)
```

morale 120 20 help → 24

morale 100 20 help → 20

morale 80 20 damage → 24



// predict multiplier

**A trader normally gives 15 oxygen.
Morale is 80. How much arrives?**

`int(15 × 80 ÷ 100) = 12 oxygen`

The Ship object carries the new state.

```
class Ship:
    def __init__(self, name, crew, oxygen, hull):
        self.name = name
        self.crew = crew
        self.oxygen = oxygen
        self.hull = hull
        self.morale = 100

    def show_status(self):
        return (
            f"Oxygen: {self.oxygen} | Hull: {self.hull} | "
            f"Morale: {self.morale}"
        )
```

Why here?

Morale belongs to this particular crew and must travel with oxygen and hull.

Choices create a feedback loop.

EVENT	MORALE CHANGE
Trader	+10
Empty space / water	+5
Asteroid field	-5
Raider	-10
Fly past	-3

Asteroids now read the ship's morale.

```
class MoraleAsteroidField:
    def __init__(self, danger_level):
        self.danger_level = danger_level
        self.base_damage = 10

    def process(self, ship):
        base = self.base_damage + (self.danger_level * 2)
        damage = int(base * (200 - ship.morale) / 100)
        ship.hull = ship.hull - damage
        ship.morale = ship.morale - 5
        return f"Asteroids: -{damage} hull, -5 morale."
```

Order matters

Calculate with the current morale first. Then apply the encounter's -5 shift.

Traders reward a confident crew.

```
class MoraleTrader:
    def __init__(self):
        self.oxygen_refill = 15

    def process(self, ship):
        refill = int(self.oxygen_refill * ship.morale / 100)
        ship.oxygen = ship.oxygen + refill
        ship.morale = ship.morale + 10
        return f"Trader: +{refill} oxygen, +10 morale."
```

Same interaction

The loop still calls `encounter.process(ship)`. Only the class knows its formula.

Add morale without losing a working game.

01

Add attribute + status

02

Update positive math

03

Update damage math

04

Add every morale shift

AFTER EACH PASS Run one fixed encounter and read the values.

Make the objects interact.

- 01 Complete all four encounter classes.
- 02 Add morale to Ship.
- 03 Apply the two fixed formulas.
- 04 Add each morale shift.
- 05 Play one careful and one reckless run.

Success looks like

The same destination produces different resource changes at morale 80 and morale 120.

Prove each branch before a full run.

120 morale

Positive outcome grows.

120 morale

Damage shrinks.

80 morale

Damage grows.

80 morale

Positive outcome shrinks.

\$ uv run game.py

Read which object Python could not find.

AttributeError

Check that self.morale is created inside Ship.__init__.

Wrong numbers

Check parentheses and whether the positive or negative formula belongs here.

```
// git verify, then save
```

Save the interacting system.

Run both play styles first. Then take the snapshot.

```
$ git status
$ git add game.py player.py
$ git commit -m "session 8:
encounter classes and
morale system"
$ git push
```

Three questions before the final voyage.

01

Why can the loop call every encounter the same way?

02

Where does morale live, and why?

03

When does the feedback loop feel fair?

Extend the system if the core is complete.

New encounter

Add Nebula, SpaceStation or Anomaly with process(ship).

Morale limits

Keep morale between 50 and 150.

Smarter scanner

Describe how current morale will affect this stop.

// session 08 complete

Your objects now change one another.
Next: make the whole game yours.

One final session: polish, final commit, showcase.

